

Memory-Centric Agentic Inference — cycles 7-9

2026-05-11

Contents

Memory-Centric Agentic Inference — cycles 7-9	1
Abstract	1
Introduction	2
Methodology	2
Results	3
Discussion	12
Conclusions and Recommendations	13
References	14
Appendix: Implementation Details	14

Memory-Centric Agentic Inference — cycles 7-9

Abstract

Cycles 7-9 extended the validated architecture package from cycles 4-6 into a second-order systems layer: trace calibration, coordination-overhead reversal, and compression/offload boundaries. The work moved from asking which architecture option is appropriate for a workload to asking what evidence a runtime must expose, when finer-grained scheduling becomes too expensive, and when compressed or summarized state preserves enough information to remain safe.

Cycle 7 built M-TRACE-1, a calibration-ready trace schema and deterministic synthetic trace v2. It records memory-object births, accesses, placements, migrations, evictions, branches, verifier events, tool outputs, semantic-cache events, provenance, invalidation, correctness sensitivity, and durable workspace state. The validated trace contains 503 synthetic events across six workloads and preserves the earlier architecture boundary: controls collapse to conventional serving, RAG maps to memory-object visibility, and code-agent, verification-heavy, and branch/merge workloads expose trajectory/DAG state. Sources: researcher session b94cef3f-d08f-4a7f-9de6-3adb677139eb, worker session 02f7c5dd-33d6-41ef-abbc-455756bfc368, auditor session e07a9416-1236-4c24-8c63-8035fe0a9224.

Cycle 8 built M-QUEUE-1, a symbolic and synthetic queueing model for metadata, policy, migration, DAG coordination, verifier synchronization, durable consistency, and preemption/checkpoint overhead. The result is a falsification model: object-aware or trajectory-aware scheduling is useful only below explicit overhead thresholds. Controls stay on conventional serving; RAG uses memory-object scheduling at low overhead but reverses to conventional serving under high object overhead; agentic workloads use trajectory/DAG scheduling at low overhead but reverse to object-level scheduling or conventional serving under high DAG or object overhead. Sources: researcher session e9534570-7fa1-434f-8ddb-8c5e9a14c997, worker session 13068dc4-bcc6-4296-90fd-8ada52f243c5, auditor session 75c154a4-08b1-41ea-8738-17a6c5c3e5ab.

Cycle 9 built M-COMP-1, a compression and offload boundary model. The main validated re-

sult is a safety boundary: compression is not just byte reduction; it is a representation choice that must preserve replay, provenance, correctness, and recovery semantics. Controls reduce to ordinary hot-state or lossless choices, RAG prefers provenance-preserving summaries for retrieved context and semantic-cache state, and agentic workloads reject unsafe lossy summaries for verifier, branch, trajectory, tool-output, and durable workspace state. The audit found and patched a moderate defect in the queue-interaction labels. After the patch, the package truthfully shows zero positive-net queue-help cases, so M-COMP-1 remains action-required for its queue-help sufficiency criterion. Sources: researcher session fefd325b-c3b2-4a6f-8af7-a8e0da80359f, worker session 0ab37c61-a633-41af-912d-d71ac412931e, auditor session 014e0070-43a1-4c82-897d-07b915ca885c.

Introduction

Cycles 1-6 established the first complete architecture proposal for memory-centric agentic inference. That proposal separated three architecture paths:

- Option A: conventional request/model/KV-centric serving.
- Option B: a memory-object-aware runtime for workloads where retrieved context, semantic-cache entries, tool outputs, provenance, and invalidation matter.
- Option C: a trajectory/DAG-aware memory fabric for branch, verifier, trajectory-log, and durable workspace state.

Cycles 7-9 did not replace that proposal. They tested what the proposal needs in order to become falsifiable and eventually calibratable against real systems.

The key shift was from static architecture selection to dynamic evidence and overhead. A memory-centric runtime cannot be evaluated only by saying that it sees more state. It also needs a trace interface that exposes the right state, a queueing model that can show when the extra coordination becomes too costly, and a compression model that distinguishes safe representation changes from unsafe information loss.

The plan of record was extended in cycle 7 with a post-architecture wave under goal G5: convert the architecture agenda into calibration-ready experiments, prototype interfaces, and second-order systems models. The new milestones were M-TRACE-1, M-QUEUE-1, M-COMP-1, M-PROTO-1, M-CALIB-1, and M-SEC-1. Cycles 7-9 completed or assessed the first three of those.

Methodology

The methodology remained artifact-driven. Researchers produced briefs, workers built inspectable models and scripts, and auditors validated the outputs against each milestone's sufficiency criteria. The reporter did not re-audit the work; this report consolidates the validated and audited record.

The three cycles form a dependency chain:

1. M-TRACE-1 defined the event interface and generated synthetic trace v2.
2. M-QUEUE-1 consumed trace v2 to reconstruct event rates, live bytes, DAG width, verifier delay, migration pressure, and durable-write pressure.
3. M-COMP-1 consumed trace v2 and queueing thresholds to score compression/offload strategies and safety failures.

All quantitative outputs in these cycles are synthetic or derived unless explicitly stated otherwise. No production traces, measured hardware latency/bandwidth constants, measured energy constants, or measured prices were introduced in cycles 7-9. REFERENCES.md was absent in the workspace, so this report cites no external numbered references.

Results

Cycle 7: Calibration-Ready Trace Schema and Synthetic Trace v2

Cycle 7 built a trace interface for memory-centric agentic inference. A trace schema is a structured event log: it defines the events and fields a runtime must emit so downstream models can reconstruct state lifetimes, reuse, placement, branch behavior, verifier delay, provenance, invalidation, and durable workspace growth.

The worker created:

- memory-centric-agentic/trace_schema.md
- memory-centric-agentic/trace_calibration_plan.md
- scripts/generate_agentic_trace_v2.py
- scripts/validate_agentic_trace_v2.py
- scripts/plot_agentic_trace_v2.py
- data/agentic_trace_events_v2.csv
- data/trace_object_lifetimes.csv
- data/trace_reuse_intervals.csv
- data/trace_branch_dag_metrics.csv
- data/trace_workload_summary.csv
- data/trace_schema_validation.csv
- data/trace_invalid_cases.csv
- data/trace_lifetime_distributions.png
- data/trace_live_bytes_by_object.png
- data/trace_branch_dag_metrics.png

The trace schema includes events for object creation, access, update, placement, migration, eviction, recomputation, branch fork/merge/discard, verifier start/result, tool calls, workspace writes, workspace compaction, semantic-cache lookup/insert, and run boundaries. It uses opaque IDs and proxy values rather than raw prompts, documents, tool outputs, or user data.

The generated trace v2 contains 503 synthetic events. It produced 66 object lifetime rows, 184 reuse interval rows, six branch/DAG metric rows, and six workload summary rows. Validation recorded zero errors on the positive trace and intended failures on negative fixtures.

The workload summaries preserve the architecture boundary from cycles 4-6:

Workload	Architecture label	Non-KV object size share	Max DAG width	Verifier results
single-turn	Option A	0.0000	0	0
chat control	Option A	0.0000	0	0
batch/offline control	Option A	0.0000	0	0
RAG	Option B	0.2485	0	0
code-agent	Option C	0.4603	2	2
loop	Option C	0.4567	3	4
verification-heavy	Option C	0.4567	3	4
multi-agent	Option C	0.5465	4	3
branch/merge	Option C	0.5465	4	3

This table is the main cycle 7 result. Controls have no branch/DAG behavior. RAG has

retrieved-context and semantic-cache reuse without trajectory dependence. Agentic workloads expose non-KV state, branch width, verifier activity, and durable state.

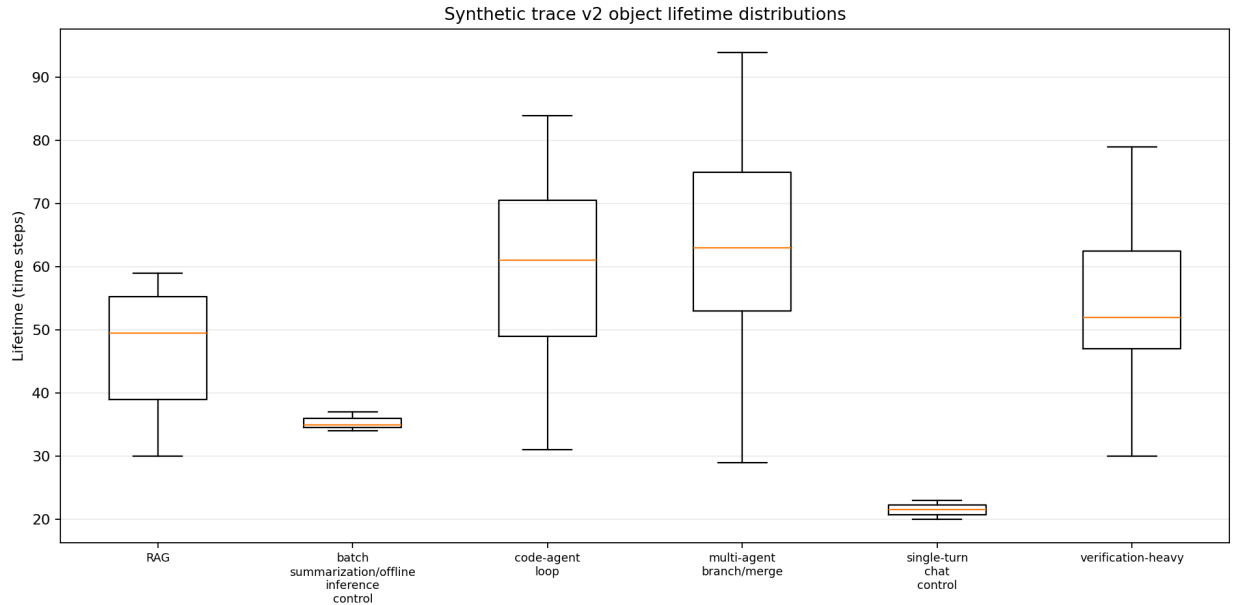


Figure 1: Cycle 7 trace lifetime distributions by object class. The figure shows that synthetic object lifetimes differ by class, giving later queueing and compression models object-level lifetime inputs rather than only request-level summaries.

The auditor found one moderate validation gap: the trace validator did not reject object events after `object_evict`. The auditor patched `scripts/validate_agentic_trace_v2.py` to add an `object_event_after_evict` check and patched the generator to add a deliberate invalid fixture. After regeneration, the positive trace still had zero errors and the invalid fixture produced 11 intended errors, including the new use-after-evict case. M-TRACE-1 was validated.

Cycle 8: Queueing and Coordination-Overhead Reversals

Cycle 8 built the overhead model that the architecture proposal needed but did not yet contain. The question was not whether Options B and C can expose useful memory state. Earlier cycles established that they can under synthetic assumptions. The cycle 8 question was when the coordination overhead of finer visibility reverses the architecture choice.

The worker created:

- `memory-centric-agentic/queueing_model.md`
- `scripts/queueing_model.wls`
- `scripts/simulate_queueing_overheads.py`
- `scripts/plot_queueing_overheads.py`
- `data/queueing_special_cases.csv`
- `data/queueing_reversal_thresholds.csv`
- `data/queueing_trace_rates.csv`
- `data/queueing_overhead_sweep.csv`
- `data/queueing_architecture_winners.csv`
- `data/queueing_failure_modes.csv`
- `data/queueing_reversal_thresholds.png`
- `data/queueing_utilization_by_workload.png`

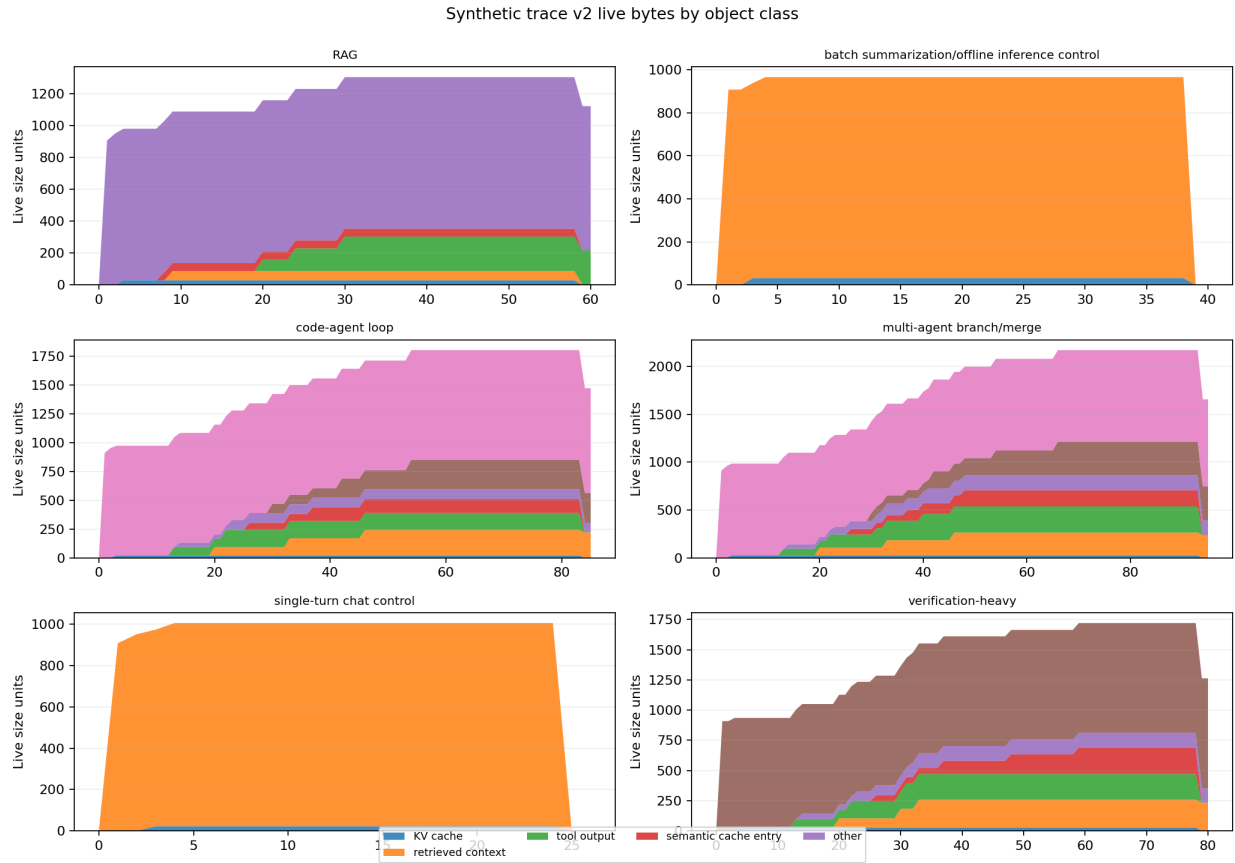


Figure 2: Cycle 7 live bytes by object class. The figure shows controls dominated by conventional state and agentic workloads carrying branch, verifier, tool-output, trajectory, and durable workspace pressure.

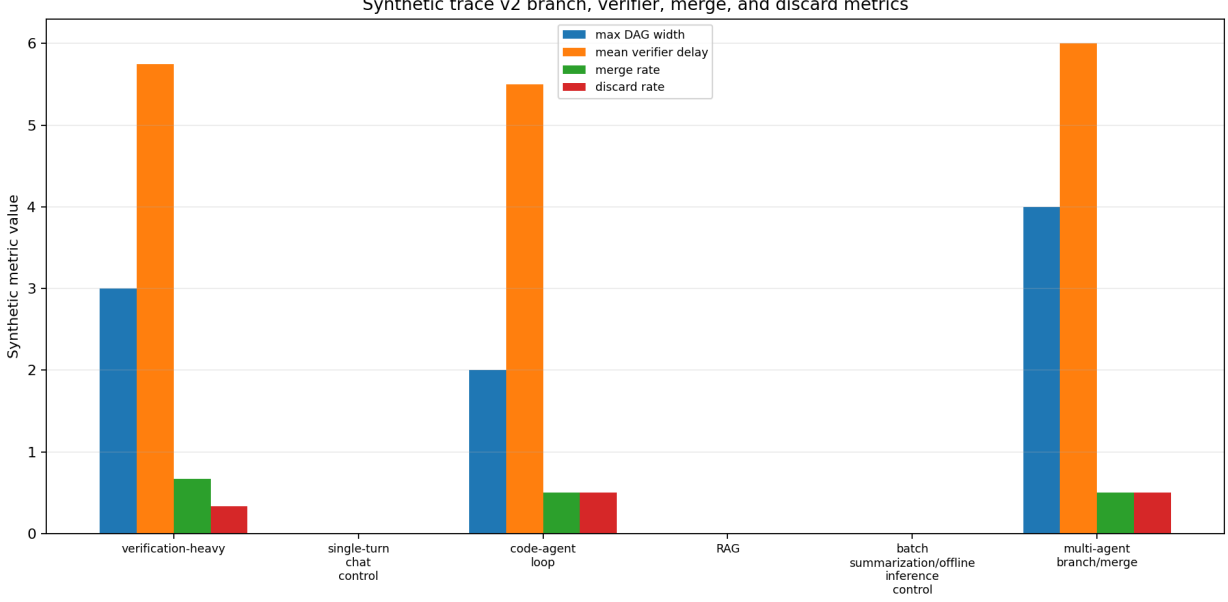


Figure 3: Cycle 7 branch and DAG metrics. The figure shows that RAG remains non-trajectory-dependent while code-agent, verification-heavy, and branch/merge workloads expose branch width, depth, and verifier delay.

- data/queueing_architecture_winner_map.png

The model uses a simple M/M/1 queueing proxy. An M/M/1 proxy is a single-server queue approximation where arrivals and service are represented by rates. It is not a production latency model. In this work it is a falsification instrument: if arrival rate approaches service rate, queue delay diverges, so an architecture option should lose if its coordination path saturates.

The model defines:

$$\rho_s = \lambda_s / \mu_s$$

$$Wq_s = \rho_s / (\mu_s - \lambda_s)$$

$$Q_s = ops_s \cdot Wq_s$$

Here, λ_s is the arrival rate for service s , μ_s is the service rate, Wq_s is queue delay, and Q_s is the queue-cost proxy after multiplying by the relevant operation count.

The architecture comparisons are expressed as threshold inequalities:

Comparison	Required condition
Option B beats Option A	Object reuse benefit exceeds registry, object-policy, and migration queues.

Comparison	Required condition
Option C beats Option B	Branch/verifier/durable benefit exceeds DAG, verifier-sync, durable-consistency, and preemption queues.
Option C beats Option A	Combined object and trajectory benefit exceeds all incremental coordination queues.

The synthetic sweep produced 92,160 rows. It modeled six workloads and seven overhead terms: registry, object policy, migration, DAG coordination, verifier synchronization, durable consistency, and preemption/checkpoint overhead.

The architecture-winner table shows the intended reversal behavior:

Workload	Low overhead	High object overhead	High DAG overhead
single-turn chat control	Option A	Option A	Option A
batch/offline control	Option A	Option A	Option A
RAG	Option B	Option A	Option B
code-agent loop	Option C	Option A	Option B
verification-heavy	Option C	Option A	Option B
multi-agent branch/merge	Option C	Option A	Option B

This is the main cycle 8 result. Memory-centric design remains conditional. Option B and Option C are useful only below explicit overhead thresholds. High object overhead collapses both RAG and agentic workloads toward Option A. High DAG overhead collapses agentic workloads from Option C to Option B, because object-level memory value remains useful while trajectory coordination becomes too expensive.

The auditor found one moderate defect. The initial winner summary computed reversal thresholds by scanning the full multidimensional sweep. That meant a reported object or DAG reversal could be caused by unrelated axes such as migration or preemption. The auditor patched `scripts/simulate_queueing_overheads.py` so object thresholds are computed along a controlled object-overhead diagonal and DAG thresholds along a controlled DAG/durable/preemption diagonal. After rerunning Wolfram, Python, plots, compilation, figure checks, `promise_check`, and `org_check`, M-QUEUE-1 was validated.

Cycle 9: Compression and Offload Boundaries

Cycle 9 tested compression and offload as representation choices. The key distinction is that compression is not automatically beneficial when it reduces bytes. A compressed or summarized object must still preserve the information needed for replay, provenance, correctness, invalidation, and recovery.

The worker created:

- `memory-centric-agentic/compression_model.md`
- `scripts/compression_model.wls`
- `scripts/evaluate_compression_strategies.py`
- `scripts/plot_compression_strategies.py`
- `data/compression_special_cases.csv`
- `data/compression_boundary_inequalities.csv`
- `data/compression_strategy_scores.csv`

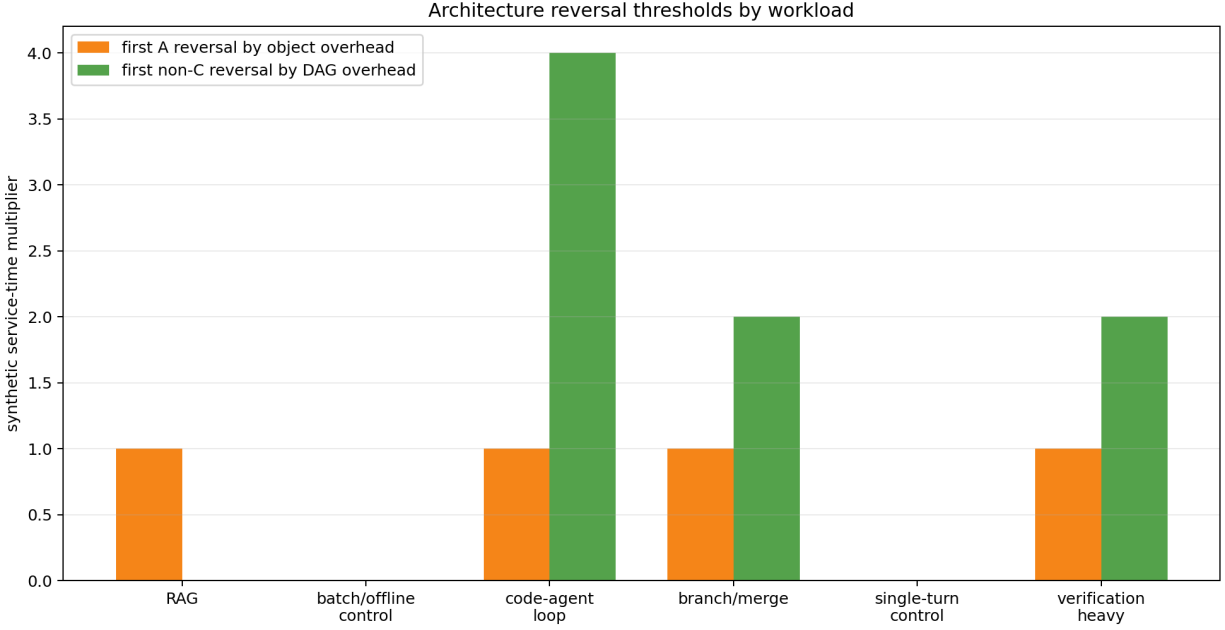


Figure 4: Cycle 8 reversal thresholds. The figure presents derived threshold expressions for when object-aware and trajectory-aware scheduling must beat their incremental coordination queues.

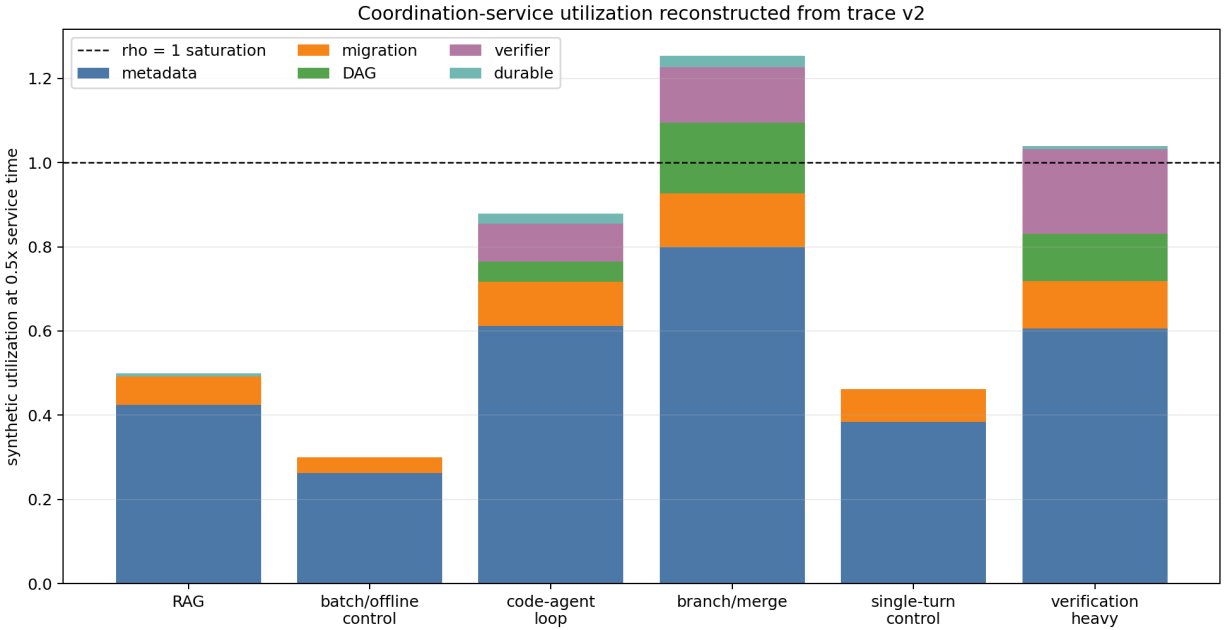


Figure 5: Cycle 8 utilization pressure by workload. The figure shows how trace-derived event rates and service assumptions produce pressure on registry, migration, DAG, verifier, durable, and preemption paths.

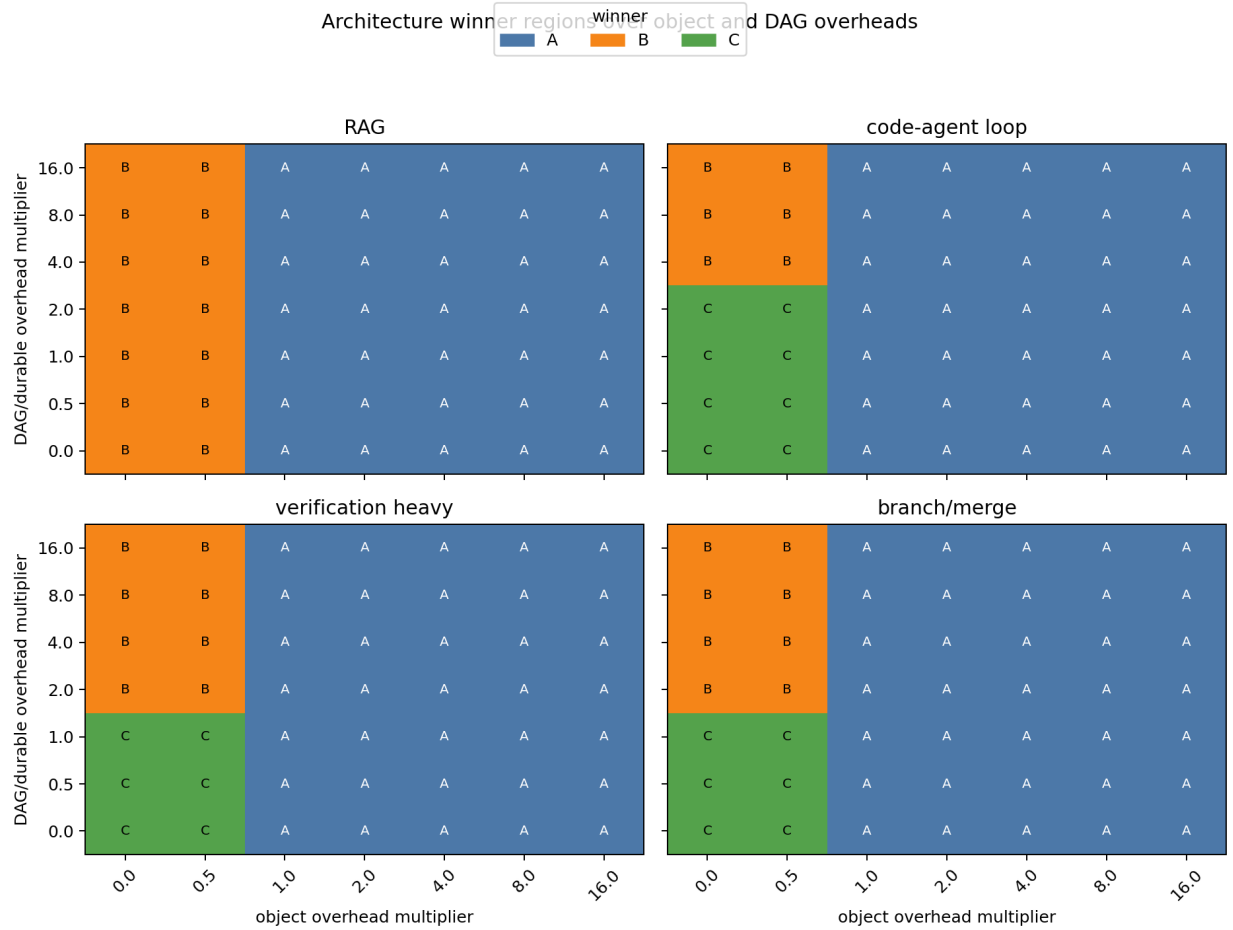


Figure 6: Cycle 8 architecture winner map. The figure shows regions where Options A, B, and C win under synthetic object-overhead and DAG-overhead sweeps.

- data/compression_best_strategy_by_object.csv
- data/compression_workload_summary.csv
- data/compression_safety_failures.csv
- data/compression_queue_interactions.csv
- data/compression_strategy_matrix.png
- data/compression_safety_vs_savings.png
- data/compression_queue_relief.png

The strategy set includes:

Strategy	Meaning
keep_hot	Keep the full object in the hot tier.
lossless_compress	Store a reversible compressed representation.
lossy_summarize	Store an irreversible summary.
summary_plus_pointer	Store a hot summary plus pointer to recover or validate the full source.
offload_full	Move the full object to a colder tier.
recompute_on_demand	Discard and rebuild later from source.

The central scoring expression is:

$$NetCompressionValue(i, s) = BytesSaved(i, s) \cdot CapacityPressure(w) + TransferAvoided(i, s) + QueueCost(i, s)$$

Unsafe lossy strategies are rejected before ranking. This hard validity gate matters because a lossy summary of verifier state, branch state, trajectory logs, tool outputs, semantic-cache entries, or durable workspace state can preserve bytes while destroying the evidence or replay path needed by the agentic run.

The workload summary shows the resulting boundary:

Workload	Best strategy counts	Boundary
single-turn chat control	keep_hot=3; lossless_compress=1	Ordinary control boundary
batch/offline control	keep_hot=2; lossless_compress=1	Ordinary control boundary
RAG	lossless_compress=3; summary_plus_pointer=3	Provenance-sensitive boundary
code-agent loop	lossless_compress=3; summary_plus_pointer=5	Agentic replay boundary
verification-heavy	lossless_compress=2; summary_plus_pointer=4	Agentic replay boundary
multi-agent branch/merge	lossless_compress=3; summary_plus_pointer=5	Agentic replay boundary

Object-level results make the pattern concrete. In RAG, retrieved context, semantic-cache entries, and tool outputs select `summary_plus_pointer`. In code-agent and branch/merge workloads,

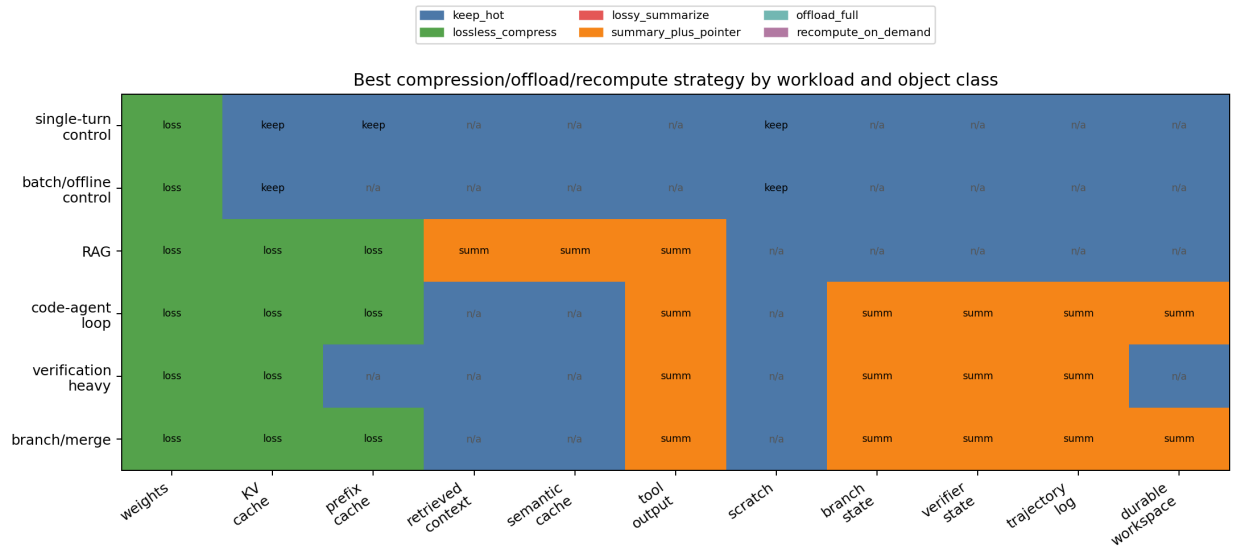


Figure 7: Cycle 9 compression strategy matrix. The figure shows the selected strategy by workload and memory-object class, separating ordinary control compression from provenance-preserving RAG and replay-preserving agentic choices.

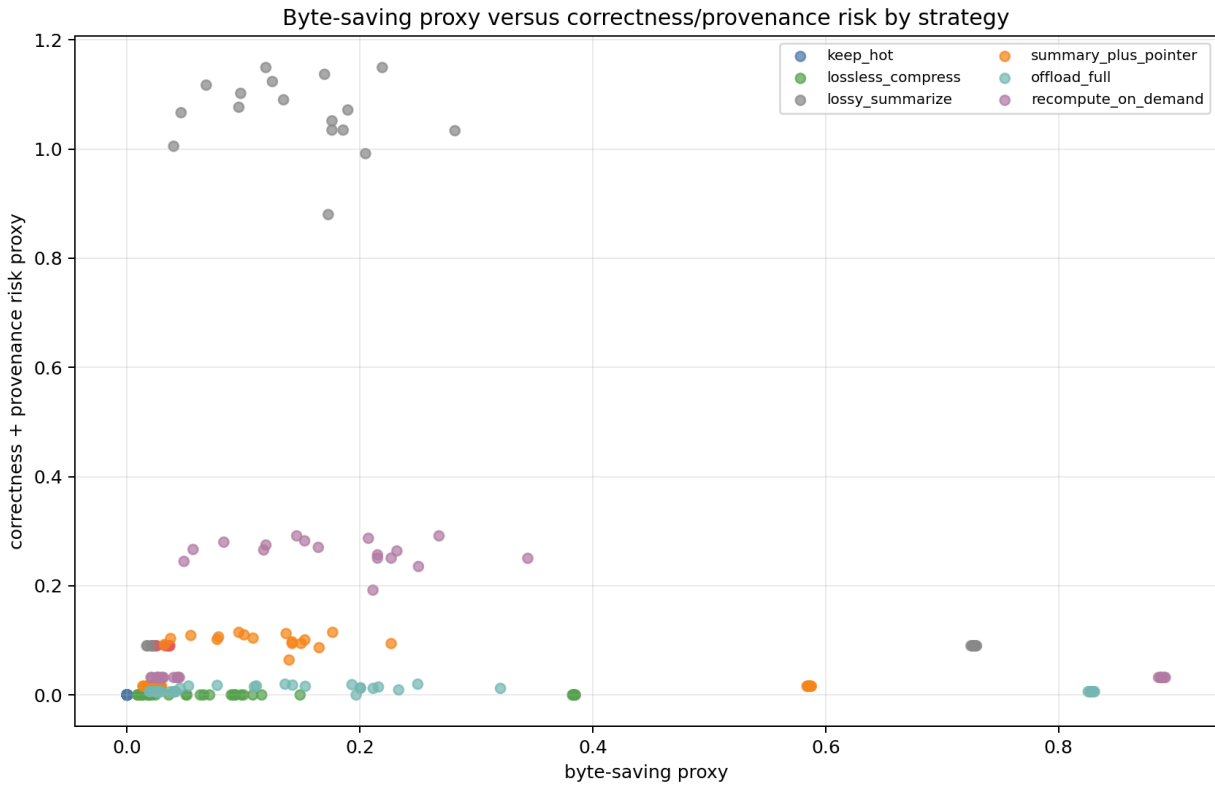


Figure 8: Cycle 9 safety versus savings. The figure shows that unsafe lossy strategies are rejected even when they would save bytes, because correctness, provenance, or replay requirements dominate.

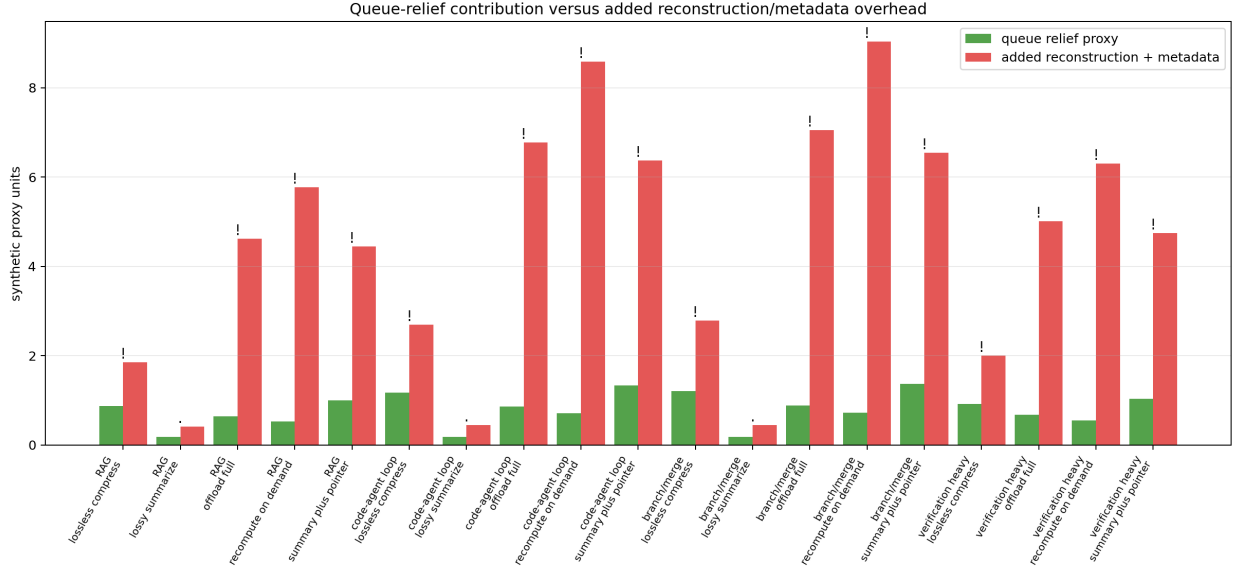


Figure 9: Cycle 9 queue relief versus added overhead. The figure shows that, under corrected labels, queue relief does not exceed added reconstruction and metadata overhead in any positive queue-help case.

branch state, durable workspace, tool output, trajectory log, and verifier state select `summary_plus_pointer`. Weights and KV-like exact state generally select `lossless_compress` or `keep_hot`.

The audit found one moderate defect. `data/compression_queue_interactions.csv` had labeled eight rows as helping avoid queue reversals even though every one had negative `net_queue_effect_proxy`. The auditor patched `scripts/evaluate_compression_strategies.py` so any `helps_*` classification requires `net_queue_effect_proxy > 0`.

After the patch, the tests passed:

- Wolfram compression model: 11 special cases and five inequalities.
- Python evaluator: 210 score rows, 35 best-strategy rows, six workload summaries, 29 safety failures.
- Plot generation: three nonblank figures.
- Python compilation: passed.
- `promise_check`: green, 37 events.
- `org_check`: green.

However, the corrected queue-interaction table has zero positive-net queue-help rows. It contains seven `local_tradeoff_only` rows and 16 `can_worsen_or_cause_reversal` rows. Therefore the safety and representation-validity result is useful, but M-COMP-1 did not meet the sufficiency criterion requiring at least one case where compression helps avoid a queueing reversal threshold. The auditor marked the milestone `action_required`, not validated.

Discussion

The cycle 7-9 results narrow the memory-centric architecture claim. The core design is still a three-level architecture, but the new work adds operational conditions:

1. A runtime must expose memory-object and trajectory variables before memory-centric policies can be evaluated.
2. Finer-grained scheduling must remain below coordination-overhead thresholds.

3. Compression must preserve the semantic boundary of each object, not just reduce bytes.

The trace schema is the strongest bridge from synthetic research to real calibration. It defines the minimum observable variables needed by later studies: object class, object size, birth and end state, reuse distance, tier placement, migration, branch identity, trajectory node, verifier delay, durable horizon, provenance ID, source version, invalidation signal, correctness sensitivity, recomputation cost, and loss cost. Without these fields, later queueing or compression conclusions would depend on hidden runtime state.

The queueing model makes the architecture falsifiable. Option C should not always win. If DAG coordination, verifier synchronization, durable consistency, or preemption queues saturate, trajectory/DAG scheduling must lose. Option B should not always win either. If object-registry, object-policy, or migration queues dominate object reuse benefit, object-level scheduling must collapse back to conventional serving. This supports a conditional architecture rather than a universal replacement for existing LLM serving.

The compression model adds a representation boundary. For controls, compression is mostly ordinary hot-state and lossless compression. For RAG, retrieved context and semantic-cache entries need provenance-preserving summaries, not ungrounded lossy summaries. For deeper agentic workloads, branch state, verifier state, trajectory logs, tool outputs, and durable workspace state need exact recovery or pointer-preserving summaries because the state carries evidence, replay, merge/discard correctness, or auditability.

The unresolved cycle 9 point is important. Compression/offload may help local capacity or transfer pressure, and it clearly clarifies safety boundaries. But under the corrected synthetic coefficients, it does not yet show a positive case where queue relief exceeds reconstruction and metadata overhead enough to avoid an M-QUEUE-1 reversal. The next worker cycle should either produce a defensible positive queue-help case through a more selective mechanism or explicitly narrow the compression claim.

Conclusions and Recommendations

Cycles 7-9 strengthen the research package by converting the architecture proposal into a more falsifiable systems model.

The validated conclusions are:

- The trace interface can represent the memory-centric variables needed by later models without storing raw user data or tool output.
- Controls collapse to conventional serving in trace, queueing, and compression outputs.
- RAG remains an object-level memory problem, driven by retrieved context, semantic-cache entries, tool outputs, provenance, and invalidation.
- Code-agent, verification-heavy, and branch/merge workloads remain trajectory/DAG problems when overhead is low enough.
- Queueing overhead can reverse the architecture choice, so memory-centric design should be conditional on measured registry, policy, migration, DAG, verifier, durable, and preemption costs.
- Unsafe lossy compression should be rejected for correctness-sensitive or replay-sensitive agentic state before score ranking.
- Compression/offload is currently supported as a safety and representation-boundary model, but not yet as a proven positive queue-reversal mitigation under the corrected synthetic setup.

The next work should focus on repairing or narrowing M-COMP-1 before treating compression as a queue-relief mechanism. A focused next cycle should revisit the interaction between

queue_relief_proxy and metadata_cost_proxy + reconstruction_cost_proxy, preferably with selective compression of high-migration or high-registry-pressure objects rather than aggregate workload-strategy labels. It should also add an assertion that all helps_* queue-interaction labels have positive net_queue_effect_proxy.

After that repair, the package is positioned for M-PROTO-1: a toy runtime prototype using trace v2 as input, implementing an object registry and placement-policy loop, and reproducing the Option A/B/C boundary under controlled overhead.

References

No external references are cited in this report. REFERENCES.md was not present in the workspace, and the cycle 7-9 artifacts explicitly used synthetic or derived quantities rather than sourced hardware constants, production traces, measured energy values, or measured prices.

Appendix: Implementation Details

Session References

Cycle	Role	Session ID	Main content
7	researcher	b94cef3f-d08f-4a7f-9de6-3adb677139eb	Defined M-TRACE-1 and post-architecture calibration wave.
7	worker	02f7c5dd-33d6-41ef-abbc-455756bfc368	Built trace schema, generator, validator, plots, and synthetic trace v2.
7	auditor	e07a9416-1236-4c24-8c63-8035fe0a9224	Patched use-after-evict validation gap and validated M-TRACE-1.
8	researcher	e9534570-7fa1-434f-8ddb-8c5e9a14c997	Defined M-QUEUE-1 queueing and coordination-overhead questions.
8	worker	13068dc4-bcc6-4296-90fd-8ada52f243c5	Built symbolic queueing model, trace-rate reconstruction, sweeps, and plots.
8	auditor	75c154a4-08b1-41ea-8738-17a6c5c3e5ab	Patched threshold-attribution defect and validated M-QUEUE-1.
9	researcher	fed325b-c3b2-4a6f-8af7-a8e0da80359f	Defined M-COMP-1 compression/offload boundary questions.

Cycle	Role	Session ID	Main content
9	worker	0ab37c61-a633-41af-912d-d71ac412931e	Built compression/offload strategy evaluator, symbolic reductions, and plots.
9	auditor	014e0070-43a1-4c82-897d-07b915ca885c	Patched misleading queue-help labels and marked M-COMP-1 action-required.

Code Organization

Cycle 7 trace files:

File	Purpose
memory-centric-agentic/trace_schema.md	Event schema for memory-object traces.
memory-centric-agentic/trace_calibration_plan.md	Runtime mapping, field status, downstream consumers, and falsification criteria.
scripts/generate_agentic_trace_v2.py	Synthetic trace generator and derived output writer.
scripts/validate_agentic_trace_v2.py	Positive-trace and invalid-fixture validator.
scripts/plot_agentic_trace_v2.py	Trace figure renderer.

Cycle 8 queueing files:

File	Purpose
memory-centric-agentic/queueing_model.md	Queueing abstraction, trace-derived variables, reversal inequalities, and special cases.
scripts/queueing_model.wls	Wolfram symbolic reductions and special-case export.
scripts/simulate_queueing_overheads.py	Trace-rate reconstruction and synthetic overhead sweep.
scripts/plot_queueing_overheads.py	Queueing figure renderer.

Cycle 9 compression files:

File	Purpose
memory-centric-agentic/compression_model.md	Compression/offload boundary model and safety rules.
scripts/compression_model.wls	Wolfram symbolic compression inequalities and special cases.
scripts/evaluate_compression_strategies.py	Strategy scoring, hard safety invalidation, and queue-interaction classification.
scripts/plot_compression_strategies.py	Compression figure renderer.

Test and Validation Results

Cycle 7 validation results:

- python3 scripts/generate_agentic_trace_v2.py: passed.
- python3 scripts/validate_agentic_trace_v2.py: passed after auditor patch.
- python3 scripts/plot_agentic_trace_v2.py: passed.
- python3 -m py_compile scripts/generate_agentic_trace_v2.py scripts/validate_agentic_trace_v2.py scripts/plot_agentic_trace_v2.py: passed.
- Figures were checked nonblank.
- promise_check: green.
- org_check: green.

Cycle 8 validation results:

- wolfram-batch -script scripts/queueing_model.wls: passed.
- python3 scripts/simulate_queueing_overheads.py: passed.
- python3 scripts/plot_queueing_overheads.py: passed.
- python3 -m py_compile scripts/simulate_queueing_overheads.py scripts/plot_queueing_overheads.py: passed.
- Figures were checked nonblank.
- promise_check: green.
- org_check: green.

Cycle 9 validation results:

- timeout 120 wolfram-batch -script scripts/compression_model.wls: passed.
- python3 scripts/evaluate_compression_strategies.py: passed after auditor patch.
- python3 scripts/plot_compression_strategies.py: passed.
- python3 -m py_compile scripts/evaluate_compression_strategies.py scripts/plot_compression_strategies.py: passed.
- Figures were checked nonblank.
- promise_check: green.
- org_check: green.
- Milestone decision: CONTINUE / action_required, because no positive queue-help case remains after correcting labels.

File Counts and Workspace Snapshot

The manifest was updated to reflect the workspace through cycle 9.

Category	Count
Scripts	20
Script lines	4,083
Markdown model/assumption/proposal files	12
CSV data/model files	43
Figures	18
Sub-topics completed or assessed	9

Key cycle 7 data row counts:

File	Rows excluding header
data/agentic_trace_events_v2.csv	503
data/trace_object_lifetimes.csv	66
data/trace_reuse_intervals.csv	184
data/trace_branch_dag_metrics.csv	6
data/trace_workload_summary.csv	6

Key cycle 8 data row counts:

File	Rows excluding header
data/queueing_special_cases.csv	11
data/queueing_reversal_thresholds.csv	4
data/queueing_trace_rates.csv	6
data/queueing_overhead_sweep.csv	92,160
data/queueing_architecture_winners.csv	6
data/queueing_failure_modes.csv	7

Key cycle 9 data row counts:

File	Rows excluding header
data/compression_special_cases.csv	11
data/compression_boundary_inequalities.csv	5
data/compression_strategy_scores.csv	210
data/compression_best_strategy_by_object.csv	35
data/compression_workload_summary.csv	6
data/compression_safety_failures.csv	29
data/compression_queue_interactions.csv	23

Figure Inventory for This Report

Figure	Dimensions	Cycle
data/trace_lifetime_distributions.png	1920 x 960	7
data/trace_live_bytes_by_object.png	2080 x 1440	7
data/trace_branch_dag_metrics.png	1920 x 960	7
data/queueing_reversal_thresholds.png	1980 x 1044	8
data/queueing_utilization_by_workload.png	1980 x 1044	8
data/queueing_architecture_winner_map.png	1890 x 1440	8
data/compression_strategy_matrix.png	2196 x 1044	9
data/compression_safety_vs_savings.png	1710 x 1116	9
data/compression_queue_relief.png	2430 x 1152	9

Cross-Reference Map

Source artifact	Consuming artifact	Relationship
data/research_agenda_ranked.csv	plan_of_record.md	Motivated post-architecture milestones under G5.
memory-centric-agentic/trace_schema.md	scripts/generate_agentic_trace_v2.py	Defined generated event fields and event types.
scripts/generate_agentic_trace_v2.py	data/agentic_trace_events_v2.csv	Produced trace v2 event stream.
data/agentic_trace_events_v2.csv	scripts/validate_agentic_trace_v2.py	Provided positive trace and invalid fixture checks.
data/agentic_trace_events_v2.csv	scripts/simulate_queueing_overheads.py	Provided event rates, migration counts, DAG metrics, verifier delays, and durable-write signals.
scripts/queueing_model.wls	data/queueing_reversal_thresholds.csv	Exported symbolic reversal thresholds used to interpret queueing sweeps.
data/queueing_architecture_winners.csv	scripts/evaluate_compression_strategies.py	Provided high-overhead winner states used in compression queue-interaction classification.
data/trace_workload_summary.csv	scripts/evaluate_compression_strategies.py	Provided workload architecture labels, object classes, provenance, correctness, and pressure inputs.
scripts/evaluate_compression_strategies.py	data/compression_queue_interactions.csv	Produced corrected local-tradeoff and reversal-worsening classifications after auditor patch.